

[WIP] PME support across various language bindings

Language	Supports Arrow	Supports Parquet	Supports Parquet Encryption	Repo Location	Notes
C++	Yes (native)	Yes	Yes (native)	Apache C++ Arrow repo	Primary implementation, full features
Python	Yes	Yes	Yes. Bindings Cython	Apache C++ Arrow repo	pyarrow full Parquet + encryption
Rust	Yes (native)	Yes (native)	Limited	Separate repo	Parquet read/write stable, encryption limited
Go	Yes (native)	Yes (native)	No	Separate repo	Encryption in internal pkg, no public API
R	Yes	Yes	No	Apache Arrow repo	No encryption support
Java	Yes (native)	Yes (native)	Yes, internal (java/parquet)	Separate repo Encrypt	Full Parquet encryption + encryption via JNI
JavaScript	Yes (native)	Yes (native)	No	Separate repo (apache/arrow-js)	No encryption support
C# (.NET)	Yes	Yes	Yes. C++ via P/invoke	Separate repo	Parquet read/write;

					encryption via P/Invoke to native C++
Ruby	Yes (Clib Wrapper)	Yes (Clib Wrapper)	No	Separate repo (red-arrow)	Basic Parquet only
MATLAB	Yes	Yes	No	MathWorks official	Read/write Parquet, no encryption

More here: [Apache Arrow: multi-language implementations](#)

Java

Apache Arrow

In the official Apache Arrow documentation

(<https://arrow.apache.org/docs/developers/java/building.html>), it is stated that the Java code can be compiled within the `arrow/java` directory. However, this folder no longer exists in the `main` branch of the main repository.

After reviewing different tags in the repository, it becomes evident that starting from version **19.0.0** (<https://github.com/apache/arrow/releases/tag/apache-arrow-19.0.0>), the Java folder was removed. The release notes for that version, published on **January 15, 2025** (<https://arrow.apache.org/release/19.0.0.html>), include relevant information such as the following comment in [issue #44831](#):

“We should use `apache/arrow-java` like `apache/arrow-go`”

This indicates that, starting with that release, the Java code is now maintained in a separate repository

Apache Arrow JAVA

`apache/arrow-java` → <https://github.com/apache/arrow-java>

This is now the official Java implementation of the Apache Arrow project. It is part of a broader ecosystem of tools that define an in-memory, columnar data format optimized for extremely fast and efficient access—especially useful for large-scale analytics and Big Data processing systems.

Code Exploration

While exploring the repository, you can find examples such as the following test file:

<https://github.com/apache/arrow-java/blob/main/dataset/src/test/java/org/apache/arrow/dataset/ParquetWriteSupport.java#L34>

This file contains unit tests related to Parquet encryption. However, several of the classes used in it are imported from other repositories, such as:

```
Java
import org.apache.parquet.avro.AvroParquetWriter;

import org.apache.parquet.hadoop.ParquetWriter;
```

This led to an exploration of the Parquet Java repository:

<https://github.com/apache/parquet-java>

There, you can find implementations of encryption-related classes, such as:

- <https://github.com/apache/parquet-java/blob/master/parquet-hadoop/src/main/java/org/apache/parquet/crypto/AesCtrEncryptor.java>
- <https://github.com/apache/parquet-java/blob/master/parquet-hadoop/src/main/java/org/apache/parquet/crypto/AesGcmEncryptor.java>

These classes are similar to the ones used in the C++ implementation of the same project. Which indicates that this project does not use any of the C code, since it has all the code implemented in Java.

C#

Within the [Apache Arrow](#) repository, specifically in the C# documentation, you can find the current [feature matrix](#). This matrix details the supported data types and in-memory manipulation. However, there is no reference to **encryption** or **decryption** functionalities in this section.

Encryption Support in ParquetSharp.

Upon further investigation, repositories that do offer encryption support were identified. The ParquetSharp package manages its own versioning scheme, which generally aligns with the version of the underlying Parquet C++ library it links to. Information about the current version and version history is available at:

- NuGet: <https://www.nuget.org/packages/ParquetSharp>
- GitHub Repository: <https://github.com/G-Research/ParquetSharp>

The ParquetSharp repository includes key classes related to encryption, such as:

- [CryptoFactory.cs](#)
- [DecryptionConfiguration.cs](#)
- [EncryptionConfiguration.cs](#)
- [KmsConnectionConfig.cs](#)

These classes among others function as interfaces and configuration containers for interacting with the encryption logic implemented in C++.

How Encryption Classes Work:

1. ``EncryptionConfiguration.cs`` and ``DecryptionConfiguration.cs``:
 - These C# classes are simple data structures that define the parameters for encryption and decryption (e.g., which columns to encrypt/decrypt, key IDs, algorithms).
 - When methods like ``ParquetWriterPropertiesBuilder.WithEncryption()`` are invoked, ParquetSharp internalizes the values from the C# ``EncryptionConfiguration`` object and serializes (or maps) them to the corresponding C++ data structures.
 - Subsequently, these C++ structures are passed to the ``DllImport``-ed functions, which are the ones that actually execute the encryption logic of the Parquet C++ library.
2. ``IKmsClient`` and Implementations (e.g., ``DictionaryKmsClient``):
 - Parquet's modular encryption requires a Key Management System (KMS) to manage master keys. ParquetSharp needs a way to interact with this KMS.
 - The ``IKmsClient`` interface is the C# abstraction for this interaction. The user (or a specific KMS provider library) must implement this interface to define how data keys are "Wrapped" (encrypted) and "Unwrapped" (decrypted) using the master keys from the KMS.
 - When ParquetSharp requires a key (e.g., when encrypting or decrypting a file), a "callback" occurs from the C++ code to the C# code using a reverse P/Invoke mechanism. The C++ function calls the ``IKmsClient`` implementation in C# provided by the user. This C# implementation communicates with the actual KMS

- (or, in the case of `DictionaryKmsClient`, performs a simple dictionary lookup).
 - The results of `WrapKey` or `UnwrapKey` (the encrypted/decrypted keys) are returned to the C++ code, which uses them to encrypt/decrypt the Parquet file data.
3. `CryptoFactory.cs`:
- This class (and others like `KmsClientFactory`) essentially act as bridges for creating encryption-related objects on the C++ side.
 - For example, there might be a `DllImport` function in C# that invokes a C++ function to "create" a native encryption factory object, and `CryptoFactory` in C# serves as a wrapper for that call.

Javascript

Similar to the C# folder, JavaScript has its own folder within the repository; however, no encryption implementation is observed within it. This folder appears to offer support for tables and vectors.

Further investigation revealed this JavaScript repository for using parquet files: <https://github.com/ironSource/parquetjs/tree/master/lib>. However, the project does not appear to be very active, given that its last commit was made a year ago, and it does not seem to support either encryption.

JavaScript does not have a direct binding to the Parquet C++ library: Unlike Python or C#, there is no standard mechanism or widely adopted binding that allows JavaScript (in Node.js or the browser) to directly invoke the functions of the Parquet C++ library.

Ruby

The Apache Arrow Ruby repo (via the [ruby/red-parquet](#) README) provides only basic Parquet read/write functionality—**no mention of encryption** [Stack Overflow](#) [GitHub](#) [GitHub](#) [GitHub](#).

Matlab

The MathWorks page on “Apache Parquet Data Type Mappings” confirms the functionality of MATLAB’s Parquet read/write via Arrow—but makes **no mention of encryption or decryption support** whatsoever.

Go

The Arrow Go module does include an **internal Parquet encryption implementation**, though it's not (yet) exposed through the high-level APIs. You can access it via the github.com/apache/arrow/go/parquet/internal/encryption package [Reddit+13pkg.go.dev+13pkg.go.dev+13](#).

Apache Arrow provides a C Data Interface (also called "C Arrow") that allows different languages—including Go—to share data using a stable ABI.

Rust

As of July 2025, **Rust Apache Arrow** has solid support for reading and writing Arrow and Parquet data, but **Parquet encryption support is still very limited or experimental** and not exposed through stable Rust APIs.

R

As of July 2025, **the official R Arrow package does not support Parquet encryption** yet. The R Arrow package provides robust read/write support for Arrow and Parquet files, but **encryption and decryption of Parquet files are not available in R**.